PRA-MLX: Query-Addressed Memory beside Prompt and Rotating KV Caches

Jorge Simão
EInnovator R&D

August 2026

Abstract

Apple Silicon changes the physical cost model of long-context inference: weights, cache arrays, and host code share unified memory. It does not remove the logical question of which retained evidence should be active. We implement an MLX adapter boundary for Progressive Retrieval Attention (PRA), measure selected-text coexistence with the MLX-LM nearest-prefix cache, and test whether a rotating sequential cache can serve as a simpler archive. On an Apple M5 with the 4-bit Qwen3-0.6B checkpoint, selected and full context recover a fixed answer in 5/5 server requests; selected context uses 365 rather than 1,577 prompt tokens. Warm requests reuse 364 and 1,576 cached tokens, respectively, while the server prompt cache grows to 0.28 GB across five distinct sequences. In a separate five-seed Python-API experiment, full sequential KV recovers the answer in 5/5 cases with 112 MB of cache. Rotating KV at 64–512 tokens recovers 0/5. Reintroducing the selected archive record reaches 2/5 only at 64 tokens and 0/5 at larger capacities. We then implement native selected K/V at every layer. Across five seeds it is bit-exact to ordinary split-cache execution, recovers 5/5 answers, and remains 5/5 when local sequential K/V rotates at 64 tokens. Thus native PRA restores archived evidence without making it visible prompt text. A follow-up profile sweep finds that the last 14 of 28 layers retain 5/5 recovery with 7.71 rather than 15.42 MiB active K/V. Persisted K/V reloads in 7.2 ms with zero logit error, and shared-memory throughput rises from 6.04 to 16.01 requests/s between concurrency one and eight. On 40 QASPER and 40 HotpotQA natural-text transport controls per condition, ordinary and native memory rank the answer correctly in every case; disabled and shuffled memory remain near chance. A larger Qwen3-1.7B natural-QA study then uses the datasets' original questions and answers. Full-precision native K/V exactly matches ordinary split-cache token F1 and answer log-probability on QASPER, HotpotQA, and 2WikiMultiHopQA. Shuffled and absent memory degrade both metrics on QASPER and HotpotQA; on 2Wiki the gold-answer log-probability degrades while free-generation F1 is noisy. Per-head int8 residency halves selected-K/V bytes and preserves this small cohort's quality, with request-local dequantization before attention. Finally, an SDK hybrid router selects four documents or paper paragraphs before native materialization. Native and ordinary consumption agree exactly on all 60 paired generations. Gold evidence recall at four is 0.348 on QASPER, 0.725 on HotpotQA, and 0.775 on 2Wiki. Routed memory outperforms shuffled memory on answer likelihood in all three datasets, but remains below oracle evidence; discovery, rather than MLX transport, is now the measured bottleneck. On the same frozen routed selections, a matched E0/E2 economic benchmark preserves all 840 output pairs across cold, warm, multi-query, and concurrency-eight schedules and removes 90.6–93.0% of visible prompt tokens. Native ingestion is cheaper than ordinary source-cache construction, but the unfused selected-cache path yields macro E2/E0 cost ratios of 1.130, 1.155, 1.148, and 1.132 across those schedules. A 1,020-request pressure sweep then holds QA quality constant while varying an eight-resource session's compact-K/V budget. Budgets below eight reload every repeated access; fitting all eight resources eliminates reloads and halves mean resolution time at a 151–168 MiB residency cost.

1 Introduction

MLX provides an efficient array framework and model runtime for Apple Silicon [1]. MLX-LM adds explicit prompt caches, rotating K/V, optional cache quantization, batching, and an OpenAI-like server. These tools make sequential context reuse practical in unified memory. PRA addresses a different object:

$$\text{large retained memory} \rightarrow \text{query-selected region} \rightarrow \text{active native detail.}$$

Unified memory may reduce copy overhead, but it does not make every retained token free to index, keep, or attend to.

This paper studies six questions. First, can selected context coexist with MLX's exact prompt cache? Second, can a bounded rotating cache plus a selected archive record recover old evidence? Third, what software boundary is needed before an MLX adapter may claim native PRA K/V rather than selected visible text? Fourth, how much native K/V can consumer-layer profiles remove without losing the controlled answer? Fifth, do persistence, request sharing, and natural-text causal controls, original benchmark questions, and quantized residency preserve the mechanism? Sixth, when the runtime itself selects evidence, can we separate routing error from native-K/V consumption error?

2 Cache Taxonomy

We keep three mechanisms distinct:

Prompt cache
: exact reusable prompt-prefix K/V, retained by MLX-LM.

Rotating cache
: bounded recent sequential K/V, with old positions evicted as the prompt advances.

PRA memory
: query-addressed non-prefix resources whose address state, native detail, and active selection have separate lifecycles.

Prompt-cache bytes are physical retained state. PRA backing may be much larger than active detail. Rotating capacity bounds direct context but does not create an address for evicted evidence.

3 Implementation

The engine-neutral namespace in `src/pra_hf/engine_memory.py` identifies a block by tenant/session scope, resource/version, token interval, consumer layer, immutable model revision, dtype/layout, materialization profile, and position policy. Its residency state machine records indexed-only, off-device, prefetching, resident, pinned, evictable, and invalid states. In MLX, the tier name denotes array residency or backing policy within unified memory; we do not mislabel it as CUDA host-to-device transfer.

`src/pra_mlx/adapter.py` exposes the HTTP server as E0. It advertises automatic prefix caching and text fallback, but no native K/V. An in-process `MLXNativeExecutor` is required before the adapter advertises E2, selected interval materialization, logical resource deltas, or explicit prompt cache handles. `src/pra_mlx/native.py` now provides that executor.

The design choice is important. Precomputing a prompt cache from selected text still makes that text a sequential prefix. A native executor must instead preserve the selected region's identity and positional policy while combining its K/V with local K/V under correct model attention.

The implementation encodes immutable resource text once and retains post-RoPE K/V in $[B, H_{kv}, T, D]$ form for every layer. A request-local cache returns *selected memory plus local sequential K/V* to each attention module and constructs a mask in which every query sees selected memory while local keys remain causal. Query positions advance from the source extent; selected text never appears in the visible prompt. The residency manager deduplicates materialization, pins blocks for request lifetime, and evicts only unpinned detail under a bounded byte budget.

For compact residency, each K and V head receives a symmetric int8 scale over its token and head-width axes. Compact arrays remain the retained object; selection dequantizes them to the model-native dtype before constructing the request-local attention cache. This version measures storage economy, not an int8 attention kernel: active attention still consumes full-precision K/V.

Consumer-layer profiles preserve the source-relative query position even on layers that omit selected detail. Such a layer receives only request-local K/V, but its RoPE offset still advances from the source extent; otherwise a profile sweep would confound memory removal with a positional-frame change. Persisted arrays carry a strict sidecar fingerprint over model and tokenizer revisions, dtype, position policy, consumer profile, and resource version. A mismatch is rejected before arrays enter attention.

Table 1: MLX-LM server smoke. TTFT is milliseconds; warm cached is the mean over repeats two through five.

Condition	Exact	Prompt	Cold TTFT	Warm TTFT	Warm cached
None	0%	24	523.6	505.2	23
Prefix	0%	333	446.4	355.2	332
Selected	100%	56	415.9	355.0	55
Prefix + selected	100%	365	432.0	352.3	364
Full context	100%	1577	565.4	351.8	1576

4 Environment

The host was an Apple M5 MacBook Pro with 10 CPU and 10 GPU cores, 16 GB unified memory, Metal 4, and macOS 26.4.1. The environment used Python 3.12.7, MLX 0.32.0, and mlx-lm 0.31.3. The model was `mlx-community/Qwen3-0.6B-4bit` at revision `73e3e38d981303bc594367cd910ea6eb48349da8`. The HTTP server retained up to 16 prompt-cache entries. It emits an explicit warning that its basic security checks are not production-grade; we treat it as an experimental deployment baseline.

The end-task study uses `mlx-community/Qwen3-1.7B-4bit`, MLX-LM 0.31.3, five seeds, and four held-out examples per seed and dataset. Evidence is oracle-scoped from dataset annotations and truncated to 384 tokens; routing quality is deliberately outside the first engine-transport experiment. A second matched-budget cohort retains every HotpotQA and 2Wiki candidate document and every QASPER paper paragraph, then routes four candidates before native materialization.

5 Server Prompt-Cache Experiment

The shared fixed task contains a stable 309-token prefix, one authoritative codeword record, twelve distractor records, and a short query. Five conditions separate no memory, prefix only, selected memory only, prefix plus selected memory, and full context. Each is repeated five times. The server reports cached tokens in the streaming usage object, and SSE events determine TTFT.

Table 1 shows correct controls: selected and full conditions are 5/5, while conditions without evidence are 0/5. Prefix plus selected memory uses 365 tokens and full context uses 1,577, again a 76.9% visible reduction. Warm selected requests reuse 364 tokens; warm full requests reuse 1,576. The server's nearest-prefix cache therefore composes cleanly with the selected-text baseline.

Warm TTFT is nearly flat, 352.3 versus 351.8 ms, because exact prompt caches remove most repeated prefill work in both conditions. This is a useful negative latency result: after a perfect repeat cache hit, reducing visible input does not improve warm TTFT on this setup. Physical retention still differs. The log shows five distinct prompt-cache sequences occupying about 0.28 GB at the end of the matrix.

6 Rotating Cache and Selected Archive

The Python-API control uses five deterministic filler permutations. One authoritative four-digit fact appears near the beginning of a roughly thousand-token prompt. We compare full sequential K/V with rotating capacities 64, 128, 256, and 512. For each rotating capacity, a second condition reintroduces the authoritative record immediately before the final question. The answer metric accepts harmless formatting but requires the correct digits.

Full sequential K/V is 5/5 with 112 MB of cache. Rotating-only is 0/5 at every capacity while reducing layer-cache storage from 112 MB to 7, 14, 28, and 56 MB. Selected archive text reaches 2/5 at 64 and 0/5 at larger capacities. Mean completion latency is 469 ms for full K/V and approximately 705–736 ms for bounded conditions. The generated outputs show chat-format and answer-path instability, not merely loss of the early fact.

This non-monotonic result should not be read as evidence that smaller cache is better. It indicates that the direct rotating-cache intervention changes more than evidence availability for this model and prompt. The selected-text fallback does not restore stable computation. A native PRA executor must retain local sequential semantics while adding selected memory separately; the next section evaluates that mechanism directly.

Peak process memory remains about 1.01–1.07 GB across these conditions despite the 7–112 MB cache range. Model and runtime allocations dominate on this small checkpoint, so layer-cache bytes are the interpretable economy metric. Larger models and memory pressure are required before process-level savings claims.

Table 2: Five-seed controlled rotating-cache experiment. Cache MB is the sum of layer-cache `nbytes`; latency is the mean completion time.

Condition	K/V tokens	Exact	Cache MB	Latency ms
Full sequential K/V	full	100%	112.0	468.9
Rotating only	64	0%	7.0	706.6
Rotating only	128	0%	14.0	730.2
Rotating only	256	0%	28.0	722.1
Rotating only	512	0%	56.0	721.2
Rotating + selected archive	64	40%	7.0	704.7
Rotating + selected archive	128	0%	14.0	736.2
Rotating + selected archive	256	0%	28.0	713.1
Rotating + selected archive	512	0%	56.0	732.2

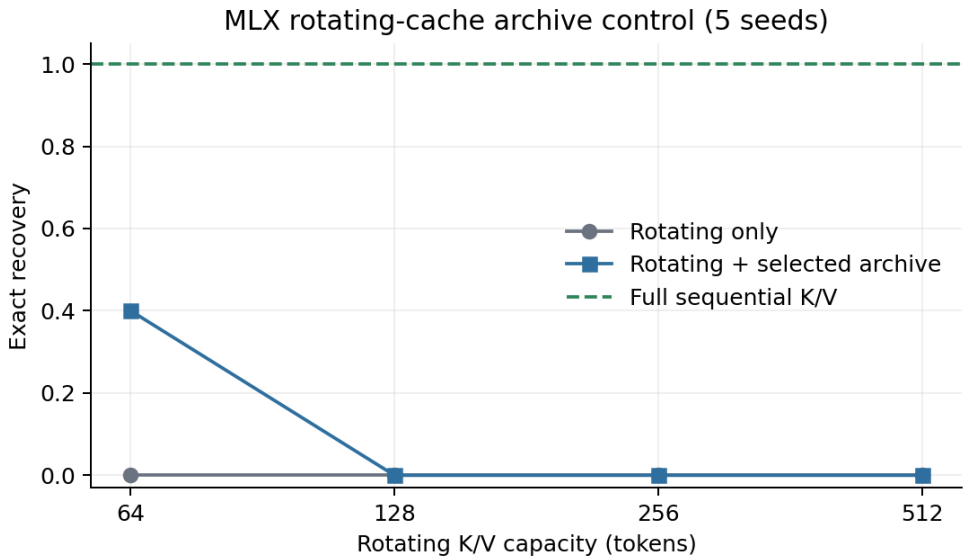

Figure 1: A bounded rotating cache is not a reliable archive in this control. Selected late text recovers two of five cases only at the smallest capacity.

7 Native Selected-K/V Experiment

The source contains 141 tokens. Ordinary split-cache and native selected-K/V produce identical logits, identical argmax tokens, and 5/5 exact recovery. Native selected execution averages 166.3 ms after a separately measured 55.4 ms one-time encoding step and retains 15.4 MB of all-layer K/V. Most importantly, replacing the local sequential cache by a 64-token rotating cache still recovers 5/5. The selected archive is therefore independent of local rotation rather than late visible text competing inside that bounded cache.

Table 4 repeats the mechanism on Llama 3.2 1B and Gemma 3 1B. Llama achieves 5/5 native and rotating-local recovery with exact split-cache logits while retaining 4.34 MB. Gemma also has zero logit error and matching argmax decisions, but both its ordinary and native conditions are 0/5 on this particular answer-format task. Gemma therefore establishes mechanism parity, not answer recovery. Together the runs cover three attention topologies and tokenizers; they are not yet natural-workload results.

8 Profiles, Persistence, and Concurrency

Table 5 exposes a sharp controlled boundary. Full native memory and the last-half profile both recover 5/5 answers. Last quarter, last four, and disabled profiles recover 0/5. Last half reduces active selected K/V from 15.42 to 7.71 MiB. Its nonzero logit difference is expected because fourteen layers no longer see the archive; exact recovery, not

Table 3: Five-seed native selected-K/V control. Latency excludes the separately reported one-time native encoding cost. Native MB is all-layer selected K/V.

Condition	Exact	Latency ms	Native MB	Max error
Ordinary split cache	100%	187.0	0.0	0
Native selected K/V	100%	166.3	15.4	0
Native + rotating local 64	100%	215.5	15.4	0

Table 4: Cross-model native selected-K/V replication. Native and rotating columns report exact recovery over five seeds; error is zero for every row.

Model	Seeds	Native exact	Rotating exact	Native MB
Qwen3 0.6B 4-bit	5	100%	100%	15.42
Llama 3.2 1B 4-bit	5	100%	100%	4.34
Gemma 3 1B 4-bit	5	0%	0%	3.58

Table 5: Five-seed consumer-layer sweep. Active MB counts selected native K/V, not model weights or request-local K/V.

Consumer profile	Layers	Exact	Active MB	Max error
all	28	100%	15.42	0
disabled	0	0%	0.00	6.188
last_4	4	0%	2.20	4.844
last_half	14	100%	7.71	4.031
last_quarter	7	0%	3.86	4.188

bit parity, is the profile criterion. This is a candidate economy profile for this fixture, not a production profile calibrated from five examples.

Table 6: Five-seed persistence and concurrent request sharing. Persist load is the mean for a 15.43 MiB selected-K/V sidecar.

Concurrency	Exact	Requests/s	Mean latency ms	Shared MB saved	Persist load ms
1	100%	6.04	164.9	0.0	7.2
2	100%	10.80	180.1	15.4	7.2
4	100%	16.98	222.4	46.3	7.2
8	100%	16.01	383.9	108.0	7.2

Persisting and reloading native arrays yields zero maximum logit error. Mean save and load times are 28.7 and 7.2 ms. Separate request-local caches share one immutable memory object: at concurrency eight this avoids 108.0 MiB of duplicate selected K/V. Aggregate throughput increases through concurrency four and then plateaus, while mean per-request latency rises from 164.9 to 383.9 ms. These are in-process controlled measurements, not queued server tail latency.

9 Natural-Text Causal Controls

The QASPER and HotpotQA builders draw natural paper or article text and place a balanced `AnswerCode yes/no` anchor among 63 source units. They test whether an answer-bearing natural-text resource survives the native transport; they do not claim unrestricted question answering. Each dataset contributes eight examples under each of five seeds, for 40 examples per condition. We rank the next-token logits for `yes` and `no`; free-form generation is retained only as a diagnostic because this small completion model often ignores the requested answer format.

Ordinary split-cache and all-layer native execution agree on all 80 examples and have identical mean margins within each dataset. Last-half execution also ranks all 80 correctly while reducing the margin. Disabled memory is 50.0% on QASPER and 52.5% on HotpotQA; shuffled memory is 50.0% on both. The causal controls therefore rule

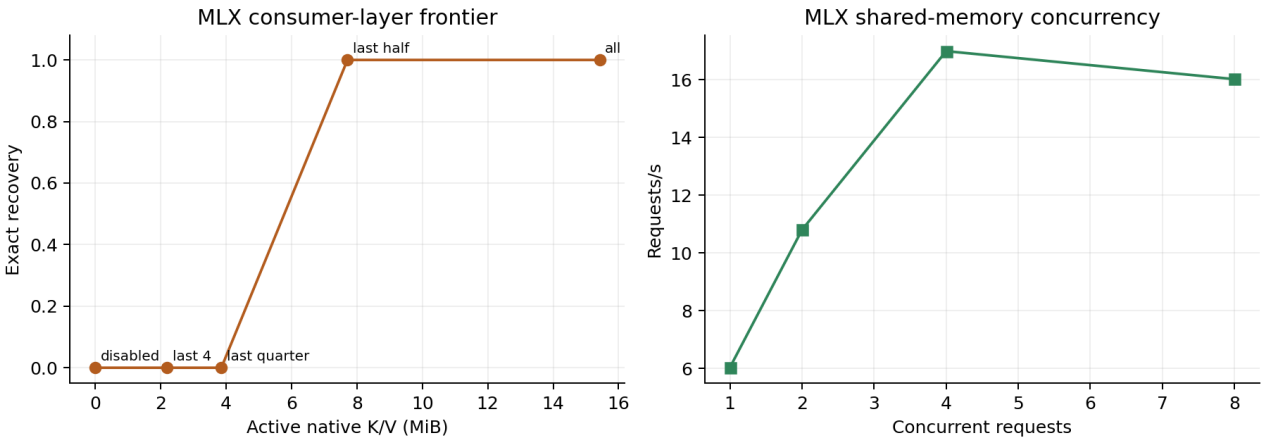

Figure 2: Left: the controlled consumer-layer quality–memory boundary. Right: shared immutable K/V raises throughput until the small Apple host saturates.

Table 7: Natural-text transport controls. Margin is the mean gold-minus-other answer logit.

Dataset	Condition	Samples	Ranked exact	Margin
qasper	native_all	40	100%	2.953
qasper	native_disabled	40	50%	0.014
qasper	native_last_half	40	100%	1.009
qasper	native_shuffled	40	50%	-0.108
qasper	ordinary_split	40	100%	2.953
hotpotqa	native_all	40	100%	2.885
hotpotqa	native_disabled	40	52%	-0.002
hotpotqa	native_last_half	40	100%	1.113
hotpotqa	native_shuffled	40	50%	0.166
hotpotqa	ordinary_split	40	100%	2.885

out query-only label bias as the source of native success for this transport task.

10 Original-Answer QA and Quantized Residency

The next experiment removes the inserted answer code. It evaluates original QASPER, HotpotQA, and 2WikiMultiHopQA questions and short gold answers. For each example, annotated evidence is placed first and natural distractor or abstract text fills the 384-token source budget. This is an oracle-evidence materialization study: it asks whether the engine transports useful native K/V, not whether PRA routing found the evidence.

Table 8: Natural-QA generation with Qwen3-1.7B-4bit over five seeds. Strict EM is zero because the model often continues after the answer; token F1 and total gold-answer log-probability retain the useful quality signal.

Dataset	Condition	n	Token F1	Gold log-prob.	Resident MiB
qasper	Ordinary split K/V	20	0.136	-28.66	0.0
qasper	Native FP K/V	20	0.136	-28.66	37.6
qasper	Native int8 resident	20	0.147	-27.73	18.8
qasper	Shuffled native K/V	20	0.057	-52.18	37.6
qasper	No memory	20	0.042	-54.53	0.0
hotpotqa	Ordinary split K/V	20	0.129	-10.95	0.0
hotpotqa	Native FP K/V	20	0.129	-10.95	42.0
hotpotqa	Native int8 resident	20	0.140	-10.21	21.0
hotpotqa	Shuffled native K/V	20	0.045	-19.48	42.0
hotpotqa	No memory	20	0.058	-15.68	0.0
2wikimultihopqa	Ordinary split K/V	20	0.059	-10.30	0.0
2wikimultihopqa	Native FP K/V	20	0.059	-10.30	42.0
2wikimultihopqa	Native int8 resident	20	0.059	-9.92	21.0
2wikimultihopqa	Shuffled native K/V	20	0.060	-16.39	42.0
2wikimultihopqa	No memory	20	0.073	-13.88	0.0

Table 8 shows exact full-precision transport parity: ordinary split-cache and native K/V have identical aggregate token F1 and gold answer log-probability on every completed dataset. On QASPER the two paths both reach 0.136 F1 and -28.66 mean log-probability; on HotpotQA both reach 0.129 and -10.95; on 2Wiki both reach 0.059 and -10.30. Shuffling memory lowers F1 to 0.057 and 0.045 on QASPER and HotpotQA and makes gold log-probability substantially worse on all three datasets. On 2Wiki, shuffled and no-memory F1 are slightly higher despite worse gold log-probability, exposing verbose generation as a noisy metric rather than supporting a quality gain. The exact transport parity and answer-likelihood controls establish source-dependent computation; broader decoding evaluation remains necessary.

Per-head int8 residency reduces mean selected storage from 37.6 to 18.8 MiB on QASPER and from 42.0 to 21.0 MiB on HotpotQA. Its aggregate F1 and gold log-probability are slightly higher in this small cohort, but this is not evidence that quantization improves quality: generation is discrete and the sample is small. The supported conclusion is non-degradation here plus a near-exact $2\times$ residency reduction. Dequantization restores model-native K/V before attention, so active materialization bytes are not reduced.

10.1 Routed evidence rather than oracle evidence

We next remove the oracle ordering. Each HotpotQA and 2Wiki article, or each QASPER paragraph, becomes a typed `AgentResource`. The product SDK's `PersistentResourceIndex` combines normalized token overlap, indexed BM25, and a deterministic signed-hash semantic control. It ranks the original question against all candidates; the top four are joined in rank order, truncated to the same 384-token budget, and encoded as native K/V. This is a parameter-free product-path baseline, not a learned retriever. Each dataset contains 20 distinct examples sampled under five seeds.

Table 9: Document routing before MLX materialization. Recall is the fraction of annotated evidence documents recovered in the first k ranks. Route time excludes one-time index construction.

Dataset	n	Candidates	R@1	R@2	R@4	Tokens	Route ms
qasper	20	46.6	0.119	0.140	0.348	370.6	11.92
hotpotqa	20	10.0	0.400	0.600	0.725	376.3	3.57
2wikimultihopqa	20	10.0	0.487	0.662	0.775	341.5	2.85

Table 9 exposes the discovery boundary. HotpotQA and 2Wiki have ten candidate documents per question and reach 0.725 and 0.775 recall@4. QASPER averages 46.6 paragraph candidates and reaches only 0.348. The selected

Table 10: Original-answer generation after routed materialization. Oracle uses annotation-scoped evidence; shuffled supplies another example's routed K/V.

Dataset	Condition	n	Token F1	Gold log-prob.
qasper	Oracle	20	0.148	-27.25
qasper	Routed	20	0.116	-38.36
qasper	Shuffled	20	0.034	-54.13
qasper	No memory	20	0.026	-60.20
hotpotqa	Oracle	20	0.129	-10.95
hotpotqa	Routed	20	0.091	-15.98
hotpotqa	Shuffled	20	0.054	-20.30
hotpotqa	No memory	20	0.058	-15.68
2wikimultihopqa	Oracle	20	0.059	-10.30
2wikimultihopqa	Routed	20	0.082	-14.31
2wikimultihopqa	Shuffled	20	0.054	-17.00
2wikimultihopqa	No memory	20	0.073	-13.88

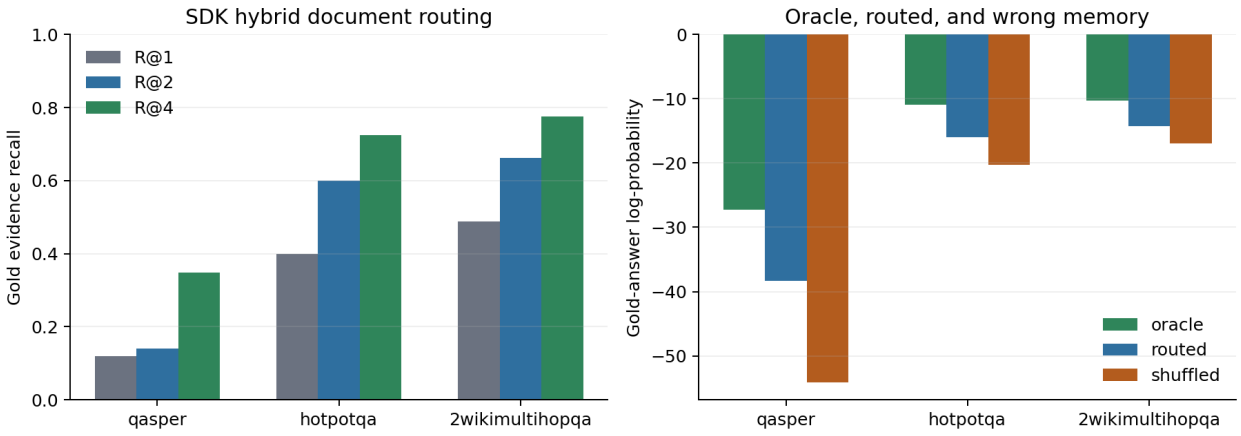

Figure 3: Left: evidence discovery improves as four candidates are admitted, but QASPER paragraph routing remains difficult. Right: routed memory carries more answer signal than shuffled memory, while the oracle gap remains.

sources consume 342–376 tokens. Mean scoring time is 2.85 ms on 2Wiki, 3.57 ms on HotpotQA, and 11.92 ms on QASPER; corresponding one-time index construction means are 6.56, 8.20, and 32.98 ms. These small exhaustive Python cohorts establish cost accounting, not a production latency claim.

The routed ordinary-cache and native-K/V paths produce the same output and exactly equal gold-answer log-probability on all 60 pairs. MLX consumption therefore adds no measured quality error after selection. Routed memory is better than shuffled memory in both F1 and gold likelihood on every dataset. It remains below oracle likelihood, however: QASPER changes from -27.25 to -38.36, HotpotQA from -10.95 to -15.98, and 2Wiki from -10.30 to -14.31. HotpotQA and 2Wiki routed likelihood also does not consistently beat the absent-memory control, despite better routed F1. Partial retrieval can be insufficient or misleading, while short free generation is noisy. The result closes the routed engine path and localizes the remaining deficit to evidence selection; it does not close a production QA-quality gate.

10.2 Matched selected-text economics

The routed selections also support a direct E0/E2 comparison without changing the evidence. For 20 examples per dataset across five seeds, E0 restores an ordinary cache snapshot of the selected source and E2 restores its native PRA cache. The question, greedy decoder, and 24-token output budget are identical. The shared schedule includes cold one-shot, two warm repeats, three deterministic query formulations over the same resource, and eight threaded requests sharing one immutable cache object.

Table 11: Matched-selection quality on the shared three-engine cohort. All parity pools 840 request pairs per engine across four schedules.

Engine	Dataset	Visible E0/E2	Reduction	Cold F1 E0/E2	Cold parity	All parity
mlx-lm	2wikimultihopqa	375/33	91.1%	0.067/0.067	100.0%	100.0%
mlx-lm	hotpotqa	415/39	90.6%	0.087/0.087	100.0%	100.0%
mlx-lm	qasper	399/28	93.0%	0.110/0.110	100.0%	100.0%
sglang-mlx	2wikimultihopqa	375/33	91.1%	0.074/0.074	100.0%	100.0%
sglang-mlx	hotpotqa	415/39	90.6%	0.084/0.084	100.0%	100.0%
sglang-mlx	qasper	399/28	93.0%	0.105/0.105	100.0%	100.0%
vllm-metal	2wikimultihopqa	378/33	91.2%	0.075/0.057	70.0%	71.1%
vllm-metal	hotpotqa	417/39	90.6%	0.074/0.080	60.0%	70.0%
vllm-metal	qasper	400/28	93.0%	0.101/0.110	55.0%	70.7%

Table 12: Macro E2/E0 execution-cost ratios; lower is better. Cold includes ingestion, concurrent is inverse requests/s, and E2 usable is milliseconds.

Engine	Cold	Warm	Multi-query	Concurrent	E2 usable ms
mlx-lm	1.130	1.155	1.148	1.132	51.2
sglang-mlx	0.976	1.136	1.140	1.122	55.0
vllm-metal	1.030	1.001	0.988	1.002	67.8

MLX-LM produces exactly the same E0 and E2 output in all 840 pairs, with equal F1 and gold evidence by construction. E2 reduces visible prompt tokens by 90.6–93.0%. Its mean time to usable context is 51.2 ms, but cold end-to-end cost is 1.130 times E0 because the unfused selected cache slows generation. Warm and multi-query ratios are 1.155 and 1.148. Under eight threaded requests, E2 reaches 88.4% of E0 throughput, an inverse cost ratio of 1.132. This is exact semantic transport across all four regimes, but not yet an optimized native-attention kernel.

Figure 5 and Table 13 validate physical lifecycle rather than theoretical byte sums. Each request pins one selected block, dequantizes it locally, generates, and releases it. With budgets $K \in \{1, 2, 4\}$, the cyclic eight-resource working set reloads all nine repeated accesses per seed. At $K = 8$, initial loads fall from 17 to eight, evictions and reloads fall to zero, and mean resolution falls from 113–119 ms to 53–57 ms. Peak compact residency rises from approximately 21 MiB at $K = 1$ to 151 MiB on QASPER and 168 MiB on HotpotQA and 2Wiki at $K = 8$.

Token F1 is unchanged across all four budgets within every dataset, and mean completion latency remains within roughly 5 ms because generation dominates the saved resolution time. The result is therefore a working-set

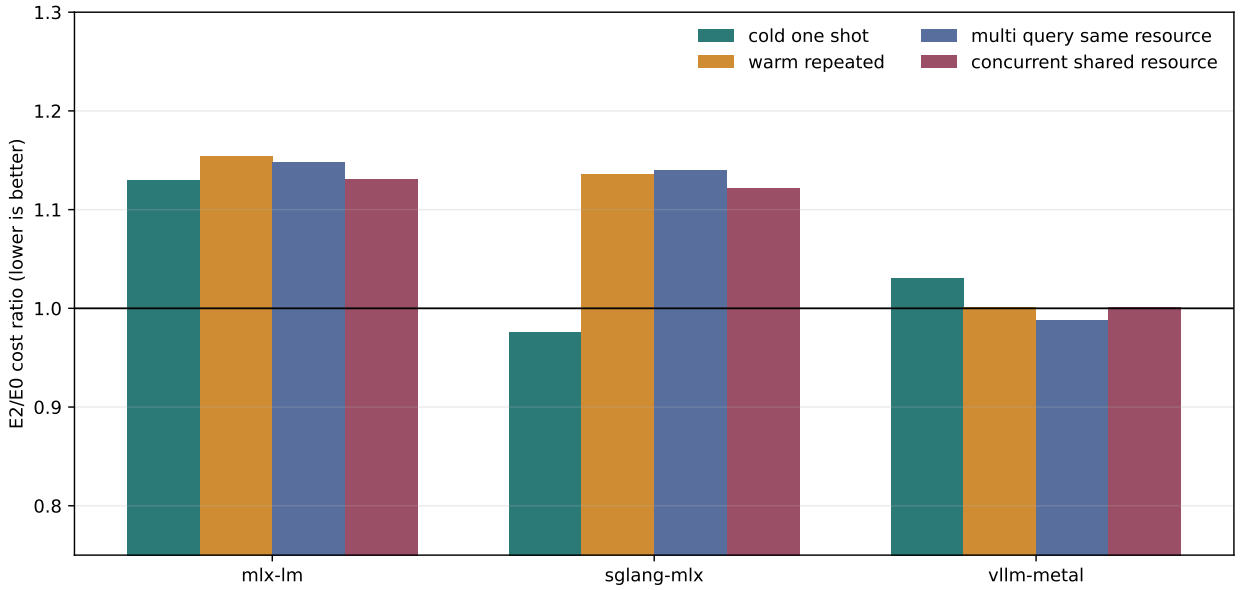

Figure 4: Macro representation cost after freezing retrieval. Lower is better; concurrent cost is inverse throughput. The current unfused MLX native cache is semantically exact but slower in every schedule.

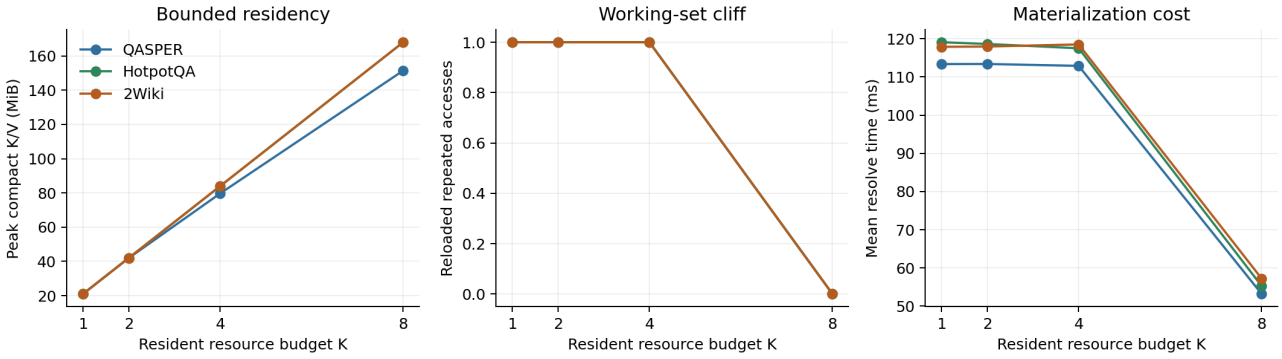

Figure 5: Five-seed compact-K/V pressure over QASPER, HotpotQA, and 2Wiki. Each seed visits eight resources twice and then revisits the first. The budget $K = 8$ is the first condition that fits the working set.

Table 13: Natural-QA residency curve. Each row contains 85 requests: 17 requests per seed over five seeds. Reload fraction divides reloads by the nine accesses after each resource's first load.

Dataset	K	n	Peak MiB	Reload frac.	Resolve ms	Token F1
QASPER	1	85	21.0	1.00	113.4	0.189
QASPER	2	85	41.9	1.00	113.4	0.189
QASPER	4	85	79.7	1.00	112.9	0.189
QASPER	8	85	151.4	0.00	53.2	0.189
HotpotQA	1	85	21.0	1.00	119.1	0.144
HotpotQA	2	85	42.0	1.00	118.6	0.144
HotpotQA	4	85	84.0	1.00	117.5	0.144
HotpotQA	8	85	168.0	0.00	55.2	0.144
2Wiki	1	85	21.0	1.00	117.9	0.049
2Wiki	2	85	42.0	1.00	117.9	0.049
2Wiki	4	85	84.0	1.00	118.5	0.049
2Wiki	8	85	168.0	0.00	57.2	0.049

threshold, not evidence that more residency generally improves answer quality or complete request latency. No pinned block is evicted and compact detail never enters an ordinary prompt or rotating cache.

11 Promotion Gates

Gate	Status	Evidence
environment	closed	MLX Metal generation and server repeat reliably
prompt-cache coexistence	smoke closed	364/365 selected tokens reused warm
rotating-cache control	controlled closed	five seeds, four capacities
native semantic parity	controlled closed	five seeds, zero logit error, 5/5 recovery
rotating plus native	controlled closed	5/5 with 64-token local cache
lazy native region	prototype closed	selected resources encode on first selection
consumer profile	controlled candidate	last 14/28 layers retain 5/5 at half K/V bytes
persistence	controlled closed	strict fingerprint; zero-error reload in 7.2 ms mean
concurrency/sharing	controlled closed	five seeds at 1–8 requests; 108 MiB avoided at eight
natural transport	controlled closed	80/80 native; disabled/shuffled near chance
natural QA transport	controlled closed	original QASPER/Hotpot/2Wiki answers with causal controls
routed natural QA	controlled partial	60/60 consumption parity; recall@4 0.348–0.775; oracle gap open
matched E0/E2 economics	controlled closed	840/840 exact outputs; all four E2/E0 cost ratios exceed one
quantized PRA detail	controlled candidate	per-head int8 halves residency; native dequantization before attention
memory pressure	controlled closed	1,020 requests; $K < 8$ thrashes, $K = 8$ eliminates reloads; OOM frontier open

12 Falsification and Limitations

Native PRA-MLX is unnecessary if selected visible text plus ordinary MLX caching matches its quality, memory, and latency on realistic long-memory workloads. The server smoke makes that baseline competitive. Conversely, the rotating control shows that bounded sequential KV is not automatically a PRA archive.

The study uses one 4-bit model for profile and natural controls, one machine, a synthetic server task, and controlled transport fixtures. The server experiment has only five repeats per condition, so p95/p99 are not reported. The natural controls contain real dataset text but an inserted fixed answer-code mechanism. The first end-task study uses original answers but oracle-scoped evidence. The routed extension has only 20 examples per condition and dataset, a parameter-free lexical/hash index, and four selected candidates. Its five seeds sample distinct examples but do not substitute for a larger cohort. Strict EM is confounded by verbose continuation. Int8 detail is restored before attention, so it saves retained bytes rather than active attention bytes. No learned native Q/K index, queued server concurrency, or OOM frontier is measured. The residency sweep varies a synthetic cyclic access schedule rather than a measured production session distribution. The matched benchmark reuses only 20 examples per dataset and does not establish end-task quality beyond proving that E2 preserves its E0 input. The rotating intervention also changes effective local context and can perturb chat-template semantics.

13 Related Work

MLX targets efficient and flexible machine learning on Apple Silicon [1]. Prompt caching and rotating K/V optimize sequential state; cache quantization reduces its physical size. Retrieval and context compression reduce visible input. PRA instead assigns stable identity to retained non-prefix memory and separates cheap addressing from active native detail. The mechanisms can be combined, but none is a semantic substitute for the others.

14 Conclusion

MLX prompt caching and selected context compose cleanly: the fixed answer is preserved with 76.9% fewer visible tokens, and repeated selected prompts reuse 364 of 365 tokens. Rotating K/V provides large layer-cache savings but fails as an archive in the five-seed control; late selected text does not reliably repair it. The in-process native executor passes the missing mechanism gate: selected non-prefix K/V is bit-exact to ordinary split-cache

execution and preserves 5/5 recovery with a rotating local cache. PRA-MLX is therefore a working native prototype. Full-precision native transport also matches ordinary split-cache quality on original-answer QA, while wrong and absent memory fail causally. Per-head int8 halves retained detail without measured degradation in the small cohort. Across a 1,020-request hard-budget sweep, sub-working-set budgets cap compact residency but reload every repeated access; fitting all eight resources removes reloads and halves resolution time without changing F1. With SDK routing, native and ordinary paths remain identical on 60/60 paired generations, but recall@4 ranges from 0.348 to 0.775 and answer likelihood remains below oracle evidence. The next quality improvement belongs in discovery, not the MLX cache transport. Queued server pressure, an OOM frontier, learned routing, and broader quality cohorts remain necessary before production claims. Matched E2 execution removes roughly 91–93% of visible tokens and preserves every output, but its unfused decode is slower. Last-half consumption and int8 residency are controlled candidates, not product profile defaults.

References

[1] A. Hannun et al. MLX: Efficient and Flexible Machine Learning on Apple Silicon. 2023.